

Reality Overlay Learning App (ROLA)

All Module Prompts

Module 1 to Module 24

Generated from available workspace PDFs, project documentation, and visible conversation prompts. Organized by module so each prompt can be reused for phased development.

Generated: September 04, 2026

Source And Recovery Note

I cannot directly open previous chat histories outside the current workspace. This PDF uses the original SRS PDF, ROLA_numbered_prompts.pdf, current visible prompts, README, and module documentation found in this project. Modules whose exact prompt text was not present are reconstructed from the implemented project scope and documentation.

Module	Title	Source
1	Android Project Foundation	Exact/recovered from current conversation request
2	Firebase Authentication	Reconstructed from SRS/project state; exact conversation prompt not available
3	ARCore Camera Scanner System	Exact/recovered from current conversation request
4	AI Object Recognition System using TensorFlow Lite	Exact/recovered from current conversation request
5	AR Floating Information Overlay	Reconstructed from SRS/project state; exact conversation prompt not available
6	Room Database and Learning History	Reconstructed from SRS/project state; exact conversation prompt not available
7	Firebase Knowledge Database and Synchronization	Reconstructed from SRS/project state; exact conversation prompt not available
8	AI Text-to-Speech Voice Explanation	Reconstructed from SRS/project state; exact conversation prompt not available
9	AI Quiz System and Gamification	Reconstructed from SRS/project state; exact conversation prompt not available
10	Testing and Deployment	Reconstructed from original SRS and project docs
11	Complete App Integration	Reconstructed from README/project integration state
12	AI Tutor Chatbot and 3D Object Visualization	Reconstructed from README/project code and docs
13	Real-Time Translation and Multilingual Learning System	Recovered from ROLA_numbered_prompts.pdf/project docs
14	AR Glasses and Wearable Device Compatibility	Recovered from ROLA_numbered_prompts.pdf/project docs
15	AI Personal Learning Engine and Adaptive Education System	Recovered from ROLA_numbered_prompts.pdf/project docs
16	Advanced Computer Vision and Scene Understanding System	Recovered from ROLA_numbered_prompts.pdf/project docs
17	Cloud AI Infrastructure and Scalable Backend Architecture	Recovered from ROLA_numbered_prompts.pdf/project docs
18	AI Knowledge Graph and Semantic Learning Network	Recovered from ROLA_numbered_prompts.pdf/project docs
19	Autonomous AI Research Assistant	Recovered from project documentation
20	Final AI Ecosystem Integration	Recovered from project documentation
21	Production Deployment	Recovered from project documentation
22	Enterprise Education Platform	Recovered from project documentation
23	Global AI Education Network	Recovered from project documentation
24	Autonomous AI Teacher and Curriculum Generator	Recovered from project documentation

Module Prompts

Module 1: Android Project Foundation

Source: Exact/recovered from current conversation request

Clean Development Prompt:

You are a senior Android software architect and Kotlin developer. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: None. This starts the Android app foundation. Now implement Module 1: Android Project Foundation. Goal: Create a complete professional Android project setup for Reality Overlay Learning App (ROLA).

Requirements:

- Native Android application using Kotlin and Android Studio latest stable version.
- Implement Clean Architecture + MVVM with app/data/local, remote, repository; domain/model, repository, usecase; presentation/screens, components, navigation, viewmodel, state; di; utils; MainActivity.kt.
- Use Jetpack Compose, Material 3, Compose Navigation, Hilt, Kotlin Coroutines, Flow, ViewModel, LiveData/StateFlow, Room, Retrofit, OkHttp, Coil, JUnit, and MockK.
- Set minimum SDK to Android 10/API 29 and target the latest Android SDK.
- Configure Gradle with plugins, dependencies, and version catalog.
- Configure AndroidManifest with CAMERA, INTERNET, ACCESS_NETWORK_STATE, Application class, MainActivity, and theme.
- Create @HiltAndroidApp ROLAApplication.
- Create Hilt modules for Network, Database, Repository, and future Firebase/ARCore/TFLite providers.
- Create navigation for Splash, Login, Home, AR Scanner, Object Detail, History, and Profile.
- Create Material 3 theme with color system, typography, and dark mode support.
- Build Splash and Home screens with professional Compose UI, cards, icons, and animations.
- Create ObjectModel and UserModel.
- Create sealed UIState with Loading, Success, and Error, backed by StateFlow in ViewModel.

Restriction / Output Note: Foundation only. Prepare the app for later AI and AR modules.

Module 2: Firebase Authentication

Source: Reconstructed from SRS/project state; exact conversation prompt not available in workspace

Clean Development Prompt:

You are an expert Android Firebase authentication engineer. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Android project foundation, MVVM, Compose UI, Hilt setup. Now implement Module 2: Firebase Authentication. Goal: Implement secure Firebase Authentication for ROLA.

Requirements:

- Integrate Firebase Authentication and required Gradle/google-services configuration.
- Create authentication architecture under data/auth, domain model/repository/usecase, and presentation/auth.
- Support user registration, login, logout, and authentication state monitoring.
- Create user profile model with userId, name, email, and profileImage.
- Use Hilt for auth repository injection.
- Use StateFlow for authentication UI state.
- Create Login/Register/Profile UI with Material 3.
- Handle invalid credentials, weak password, network failure, and unauthenticated state.
- Prepare Firestore user profile sync for later modules.
- Keep API keys and Firebase config secure through environment/project configuration.

Restriction / Output Note: Do not implement AR, ML, overlay, or history features in this module.

Module 3: ARCore Camera Scanner System

Source: Exact/recovered from current conversation request

Clean Development Prompt:

You are an expert Android AR developer. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Android setup, MVVM, Compose UI, Firebase Authentication. Now implement Module 3: ARCore Camera Scanner System. Goal: Create the ARCore camera scanner foundation.

Requirements:

- Integrate Google ARCore into the Android project.
- Create AR scanning feature so users can open camera scanner screen.
- Handle camera permission properly.
- Start AR session automatically.
- Detect real-world surfaces.
- Create and track AR anchors.
- Use ARCore, Sceneform/Filament rendering engine, Kotlin, and MVVM.
- Create presentation/scanner with ARScannerScreen.kt, ARScannerViewModel.kt, ScannerUiState.kt.
- Create data/ar with ARSessionManager.kt and AnchorRepository.kt.
- Create domain/model/DetectedObject.kt.
- ARSessionManager initializes session, checks compatibility, manages frame updates, plane detection, and anchors.
- Enable horizontal and vertical plane detection and environmental lighting.
- Create full-screen AR preview, scanning animation overlay, scan button, and status indicator.
- Handle unsupported device, permission denied, session failure, and lifecycle interruption.

Restriction / Output Note: Do not implement ML object recognition yet. Only complete AR scanning foundation.

Module 4: AI Object Recognition System using TensorFlow Lite

Source: Exact/recovered from current conversation request

Clean Development Prompt:

You are an expert Android AI + Machine Learning engineer. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Android setup, MVVM, Compose UI, Firebase Authentication, ARCore camera scanner foundation. Now implement Module 4: AI Object Recognition System using TensorFlow Lite. Goal: Recognize real-world objects from AR camera frames and return object names with confidence scores.

Requirements:

- Integrate TensorFlow Lite Interpreter and TensorFlow Lite Support Library.
- Add ML Kit Vision only if required.
- Create data/ml: TensorFlowLiteClassifier.kt, ModelLoader.kt, ImageProcessor.kt, RecognitionRepository.kt.
- Create domain/model/RecognitionResult.kt.
- Create presentation/recognition: RecognitionViewModel.kt, RecognitionUiState.kt, RecognitionResultCard.kt.
- Load .tflite model from assets.
- Prepare input image tensor, run inference, process output probabilities, and return top detected objects.
- Convert AR camera frames into Bitmap, resize to model input size, normalize pixels, and convert to TensorImage.
- Connect AR camera frame -> image processing -> TFLite model -> prediction result -> ViewModel -> Compose UI.
- RecognitionViewModel receives frames, starts inference, maintains state, handles loading/errors, and prepares data for future AR overlay.
- Display Scanning or Object Detected with name, scientific information loading, and confidence percentage.
- Optimize with asynchronous inference, frame skipping, background dispatcher, and no processing every frame.
- Handle missing model, invalid image, model loading failure, low confidence, and performance limitation.

Restriction / Output Note: Do not implement AR information overlay yet. Only complete AI object detection and recognition.

Module 5: AR Floating Information Overlay

Source: Reconstructed from SRS/project state; exact conversation prompt not available in workspace

Clean Development Prompt:

You are an expert Android AR UI and spatial interface developer. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: ARCore scanner and TensorFlow Lite recognition. Now implement Module 5: AR Floating Information Overlay. Goal: Display educational object information as floating AR panels anchored to recognized objects.

Requirements:

- Create overlay architecture under presentation/overlay, data/overlay, and domain/model as needed.
- Create AR information panel model containing object name, category, scientificName, description, uses, facts, confidence, and anchor reference.
- Bind recognition results to existing AR anchors.
- Render floating AR panels using SceneView/Filament compatible nodes or Compose overlay surfaces.
- Support readable panel layout, close/minimize action, confidence badge, and loading state for knowledge lookup.
- Handle anchor lost, low tracking quality, missing object content, and repeated detections.
- Keep AR frame rate smooth by avoiding heavy work on render thread.
- Prepare hooks for translation and voice modules.

Restriction / Output Note: Only implement AR educational display. Do not add TTS, history, or quiz logic here.

Module 6: Room Database and Learning History

Source: Reconstructed from SRS/project state; exact conversation prompt not available in workspace

Clean Development Prompt:

You are an expert Android local data and Room database engineer. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: AR scanning, object recognition, AR overlays. Now implement Module 6: Room Database and Learning History. Goal: Store scanned objects and learning history locally with offline-first architecture.

Requirements:

- Create Room entities for users, learning objects, and scan history.
- Add ScanHistory model with scanId, userId, objectId, timestamp, confidence, imageUrl/thumbnail, and learning status.
- Create DAO interfaces and repository implementations.
- Expose scan history through Flow and StateFlow-backed ViewModels.
- Create History screen with empty, loading, success, and error states.
- Allow viewing object details from history.
- Prepare migrations and database versioning.
- Handle duplicate scans, offline mode, missing user, and database errors.
- Keep repository pattern and Clean Architecture boundaries.

Restriction / Output Note: Do not implement Firestore sync yet unless only preparing sync flags.

Module 7: Firebase Knowledge Database and Synchronization

Source: Reconstructed from SRS/project state; exact conversation prompt not available in workspace

Clean Development Prompt:

You are an expert Firebase and offline-first Android engineer. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Room local database and learning history. Now implement Module 7: Firebase Knowledge Database and Synchronization. Goal: Sync object knowledge, scan history, and user learning data between Room and Firebase Firestore.

Requirements:

- Add Firestore collections for users, objects, scanHistory, and knowledge content.
- Create sync repository/manager for local-to-cloud and cloud-to-local updates.
- Use Firebase Authentication userId for ownership and access control.
- Support offline queue, retry, conflict handling, and sync status indicators.
- Prepare object knowledge exploration from Firestore-backed content.
- Implement repository methods for fetching educational object info.
- Update Firestore rules and indexes for secure reads/writes.
- Handle network unavailable, permission denied, stale data, and sync failure.

Restriction / Output Note: Do not implement quiz, voice, or translation here.

Module 8: AI Text-to-Speech Voice Explanation

Source: Reconstructed from SRS/project state; exact conversation prompt not available in workspace

Clean Development Prompt:

You are an expert Android accessibility and Text-to-Speech engineer. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: AR overlays, local history, Firebase knowledge database. Now implement Module 8: AI Text-to-Speech Voice Explanation. Goal: Provide spoken educational explanations for recognized objects.

Requirements:

- Create voice architecture under presentation/voice, data/voice, and domain/model.
- Use Android TextToSpeech engine for object descriptions and AI tutor responses.
- Create VoiceExplanationManager/Repository to initialize, speak, pause/stop, and release TTS safely.
- Support speech rate, pitch, queue mode, and locale selection.
- Add Compose playback controls for listen, pause/stop, replay, and status.
- Integrate with object detail and AR overlay text.
- Handle missing TTS engine, unsupported locale, initialization failure, empty text, and lifecycle cleanup.
- Prepare multilingual voice output for Module 13.

Restriction / Output Note: Do not implement translation in this module.

Module 9: AI Quiz System and Gamification

Source: Reconstructed from SRS/project state; exact conversation prompt not available in workspace

Clean Development Prompt:

You are an expert Android education, assessment, and gamification engineer. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Object knowledge, history, voice explanation. Now implement Module 9: AI Quiz System and Gamification. Goal: Generate quizzes from learned object content and track learner progress.

Requirements:

- Create quiz architecture under presentation/quiz, data/quiz, and domain/model.
- Create Quiz, QuizQuestion, QuizAnswer, QuizResult, Achievement, and progress models.
- Generate object-based questions from educational content.
- Create Material 3 quiz screens with question navigation, scoring, review, and feedback.
- Persist quiz definitions and results locally with Room.
- Prepare Firestore sync for quiz results.
- Add badges, streaks, XP, levels, or achievements.
- Handle empty knowledge content, incomplete quiz, retry, incorrect answers, and offline state.
- Expose results for adaptive learning module.

Restriction / Output Note: Do not implement adaptive personalization yet beyond storing useful signals.

Module 10: Testing and Deployment

Source: Reconstructed from original SRS and project docs

Clean Development Prompt:

You are an expert Android QA, release, and DevOps engineer. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Core learner modules through quiz and gamification. Now implement Module 10: Testing and Deployment. Goal: Prepare ROLA for reliable testing and deployment.

Requirements:

- Add unit tests for repositories, ViewModels, use cases, and utility classes.
- Add UI smoke tests for Compose screens.
- Add AR device testing plan for ARCore-supported devices.
- Add Firebase emulator/security rule testing plan.
- Configure CI workflows for build, test, and release.
- Prepare release signing outside source control.
- Document Android Studio setup, SDK setup, Firebase config, model asset placement, and production release process.
- Validate performance, security, compatibility, and usability requirements.

Restriction / Output Note: Do not add unrelated product features in this module.

Module 11: Complete App Integration

Source: Reconstructed from README/project integration state

Clean Development Prompt:

You are an expert Android system integration architect. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Foundation, auth, AR scanner, recognition, overlays, TTS, history, database, sync, search, quiz, gamification. Now implement Module 11: Complete App Integration. Goal: Connect the completed ROLA learning modules into one coherent learner journey.

Requirements:

- Create shared navigation paths between scanner, object detail, overlay, voice, history, quiz, profile, and search.
- Unify data flow from AR scan to recognition to knowledge lookup to overlay to voice to history to quiz.
- Create shared app-level UI state and error handling conventions.
- Ensure modules communicate through domain models and repositories rather than direct presentation coupling.
- Add integration documentation and test scenarios.
- Keep feature modules independently maintainable.

Restriction / Output Note: Do not replace existing module implementations; integrate them cleanly.

Module 12: AI Tutor Chatbot and 3D Object Visualization

Source: Reconstructed from README/project code and docs

Clean Development Prompt:

You are an expert Android AI tutor and 3D visualization engineer. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Integrated AR learning flow, quiz, voice, history, knowledge database. Now implement Module 12: AI Tutor Chatbot and 3D Object Visualization. Goal: Add follow-up AI learning conversation and interactive 3D object exploration.

Requirements:

- Create AI tutor architecture under presentation/chatbot, data/chatbot, and domain/model.
- Build tutor responses grounded in object knowledge, scan history, quiz results, and learning level.
- Cache chat messages locally and prepare backend LLM integration through a trusted server.
- Create Compose chat UI with suggestions and loading/error states.
- Create 3D visualization architecture under presentation/visualization and data/model3d.
- Load GLB/GLTF model metadata from local fallback or cloud model descriptors.
- Render interactive 3D models with SceneView/Filament and AR placement preparation.
- Provide controls for rotate, scale, reset, and model status.
- Handle missing models, invalid model files, network errors, and performance limits.

Restriction / Output Note: Do not expose raw LLM API keys inside the Android app.

Module 13: Real-Time Translation and Multilingual Learning System

Source: Recovered from ROLA_numbered_prompts.pdf/project docs

Clean Development Prompt:

You are an expert Android AI engineer specializing in multilingual AI systems, machine translation, language processing, Kotlin, Jetpack Compose, and Firebase/Room integration. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Previous modules completed as prerequisites. Now implement Module 13: Real-Time Translation and Multilingual Learning System. Goal: Enable users to view and listen to educational object information in multiple languages.

Requirements:

- Create translation architecture under presentation/translation, data/translation, and domain/model.
- Implement translation service for object information, AI tutor response, quiz, and voice explanation translation.
- Support English, Bengali, Spanish, French, German, Chinese, and Arabic with easy expansion.
- Create Language model with languageCode, languageName, nativeName, and supportedVoice.
- Implement translateText(), detectLanguage(), getSupportedLanguages(), translateObjectInformation(), translateTutorResponse(), translateQuizText(), and voice locale helpers.
- Create Material 3 language selection UI.
- Integrate translated object information with AR overlay, TTS, AI Tutor, and Quiz.
- Implement offline translation cache, local language packs, Room entities, Firestore sync, locale fallback, latency optimization, and security.

Restriction / Output Note: Do not modify existing AR scanning, ML recognition, or 3D visualization modules.

Module 14: AR Glasses and Wearable Device Compatibility

Source: Recovered from ROLA_numbered_prompts.pdf/project docs

Clean Development Prompt:

You are an expert Extended Reality software architect specializing in Android ARCore, wearable computing, AR glasses, spatial computing, and hands-free UX. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Previous modules completed as prerequisites. Now implement Module 14: AR Glasses and Wearable Device Compatibility. Goal: Prepare ROLA architecture for future integration with AR glasses and wearable XR devices.

Requirements:

- Create wearable architecture under presentation/wearable, data/wearable, and domain/model.
- Design hardware abstraction layer for AR glasses, smart glasses, XR headsets, and future wearables.
- Create WearableDevice model with deviceId, deviceName, manufacturer, connectionType, batteryLevel, displayCapability, and status.
- Implement Bluetooth/WiFi discovery, connection, and monitoring states.
- Prepare data flow: wearable camera -> object recognition -> learning information -> AR display.
- Prepare voice commands, hand tracking, touch gestures, controllers, eye tracking, spatial mapping, and depth APIs.
- Adapt UI for small field-of-view transparent displays.
- Add battery optimization, secure pairing, encrypted communication, privacy, and simulation tests.

Restriction / Output Note: Do not replace existing mobile AR functionality.

Module 15: AI Personal Learning Engine and Adaptive Education System

Source: Recovered from ROLA_numbered_prompts.pdf/project docs

Clean Development Prompt:

You are an expert AI education system architect specializing in adaptive learning, personalization, educational analytics, Kotlin, Room, Firebase, and MVVM. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Previous modules completed as prerequisites. Now implement Module 15: AI Personal Learning Engine and Adaptive Education System. Goal: Create an intelligent learning system that analyzes user behavior and recommends personalized learning.

Requirements:

- Create adaptive architecture under presentation/adaptive, data/adaptive, and domain/model.
- Track scanned objects, explored categories, quiz scores, learning time, searches, difficult concepts, preferred language, and goals.
- Create LearningProfile with userId, totalObjectsLearned, averageQuizScore, favoriteCategories, weakAreas, learningLevel, and recommendation fields.
- Analyze scans, quiz results, search history, and chatbot interactions.
- Detect strong topics, weak topics, knowledge gaps, and learning preferences.
- Generate AI recommendations and personalized roadmap.
- Adapt quiz difficulty, explanation complexity, and suggestions for Beginner, Intermediate, and Advanced learners.
- Create dashboard, Room entities, Firebase collections, privacy controls, background processing, and tests.

Restriction / Output Note: Do not modify existing AR, ML recognition, or visualization modules.

Module 16: Advanced Computer Vision and Scene Understanding System

Source: Recovered from ROLA_numbered_prompts.pdf/project docs

Clean Development Prompt:

You are an expert Computer Vision AI engineer specializing in real-time object detection, scene understanding, mobile AI optimization, TensorFlow Lite, and ARCore. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Previous modules completed as prerequisites. Now implement Module 16: Advanced Computer Vision and Scene Understanding System. Goal: Upgrade object recognition into a scene-aware multi-object vision system.

Requirements:

- Create advanced vision architecture under presentation/vision, data/vision, and domain/model.
- Support multiple object detection, tracking, classification, bounding boxes, and confidence scoring.
- Prepare EfficientDet Lite, MobileNet SSD, YOLO mobile, and optimized TensorFlow Lite detection models.
- Maintain mobile-optimized low latency and battery efficiency.
- Track object identity across frames and smooth AR overlay movement.
- Analyze scenes such as kitchens/labs and relationships such as Plant + Pot + Soil.
- Provide contextual educational content, safety information, and scientific explanations.
- Create Vision UI, support offline detection/cache, add DetectedSceneEntity, privacy, and tests.

Restriction / Output Note: Do not replace existing recognition system; upgrade it.

Module 17: Cloud AI Infrastructure and Scalable Backend Architecture

Source: Recovered from ROLA_numbered_prompts.pdf/project docs

Clean Development Prompt:

You are an expert cloud architect specializing in scalable mobile backends, AI infrastructure, Firebase, serverless systems, and secure APIs. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Previous modules completed as prerequisites. Now implement Module 17: Cloud AI Infrastructure and Scalable Backend Architecture. Goal: Transform ROLA into a scalable cloud-powered educational platform.

Requirements:

- Create backend folders for authentication, user-management, object-knowledge, ai-services, analytics, recommendation, storage, admin, and API gateway.
- Design Mobile Application -> API Gateway -> Backend Services -> Cloud Database -> AI Services.
- Implement User, Object Knowledge, AI, Analytics, Storage, and Admin services.
- Improve Firebase Authentication, Firestore, Storage, and Cloud Functions architecture.
- Use collections users, objects, scanHistory, quizResults, learningProfiles, translations, 3DModels, aiModels, and analytics.
- Store images, 3D models, AI models, and educational resources with CDN/security/versioning.
- Prepare REST/GraphQL APIs, analytics pipeline, RBAC, encryption, rules, indexes, monitoring, and scalability tests.

Restriction / Output Note: Do not modify existing mobile features.

Module 18: AI Knowledge Graph and Semantic Learning Network

Source: Recovered from ROLA_numbered_prompts.pdf/project docs

Clean Development Prompt:

You are an expert AI knowledge engineer specializing in knowledge graphs, semantic networks, ontology design, graph databases, and educational AI. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Previous modules completed as prerequisites. Now implement Module 18: AI Knowledge Graph and Semantic Learning Network. Goal: Create a semantic knowledge system that understands relationships between objects, concepts, categories, and applications.

Requirements:

- Create data/knowledgegraph with repository, graph database manager, relationship engine, and semantic search.
- Create KnowledgeNode, KnowledgeRelation, ConceptGraph, and LearningPath models.
- Support node types Object, Material, Scientific Concept, Process, Application, Theory, and Technology.
- Model relationships is_a, made_of, used_for, related_to, part_of, causes, discovered_by, and applied_in.
- Build semantic search such as electricity -> conductors -> copper -> circuits.
- Support Firebase-compatible local graph caching and graph visualization UI.
- Integrate AI Tutor, Search, Adaptive Learning, admin/AI expansion, rule-based reasoning, optimization, verified content, and tests.

Restriction / Output Note: Do not modify existing AR scanning or ML recognition modules.

Module 19: Autonomous AI Research Assistant

Source: Recovered from project documentation

Clean Development Prompt:

You are an expert AI research automation and educational content verification architect. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Previous modules completed as prerequisites. Now implement Module 19: Autonomous AI Research Assistant. Goal: Create an autonomous research assistant that expands and verifies educational knowledge.

Requirements:

- Create research architecture with ResearchRepository, ResearchAgent, KnowledgeCollector, ContentAnalyzer, VerificationEngine, SourceManager, and content generator.
- Manage research tasks, scientific sources, knowledge updates, learning materials, and content versions.
- Generate beginner/advanced explanations, quiz prompts, summaries, and flashcards.
- Require verification for source reliability, duplicate risk, scientific consistency, contradiction risk, and moderation.
- Provide admin research dashboard with pending updates and approval flow.
- Bridge approved/generated materials into knowledge graph and AI Tutor context.
- Prepare Room/Firestore structures, tests, and future scheduled research refreshes.

Restriction / Output Note: Do not auto-publish unverified educational content.

Module 20: Final AI Ecosystem Integration

Source: Recovered from project documentation

Clean Development Prompt:

You are an expert AI platform architect for Android, AR, cloud, analytics, and educational systems. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Previous modules completed as prerequisites. Now implement Module 20: Final AI Ecosystem Integration. Goal: Unify all ROLA AI, AR, learning, cloud, and analytics modules into one ecosystem.

Requirements:

- Create AIOrchestrator, AgentManager, ContextManager, DecisionEngine, LearningExperienceManager, EcosystemAnalyticsManager, and InstitutionSupportManager.
- Coordinate Vision, Knowledge, Tutor, Learning, Research, Analytics, Translation, and Quiz agents.
- Implement unified session flow: object detection -> knowledge retrieval -> AR visualization -> AI explanation -> quiz -> progress -> recommendation.
- Connect database tables/collections for users, objects, scanHistory, quizzes, chat, recommendations, graph, research, analytics, translations, and 3D models.
- Add security checklist, testing checklist, production deployment strategy, and future roadmap.

Restriction / Output Note: Do not break existing module boundaries.

Module 21: Production Deployment

Source: Recovered from project documentation

Clean Development Prompt:

You are an expert Android release, Firebase, CI/CD, security, and Google Play deployment engineer. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Previous modules completed as prerequisites. Now implement Module 21: Production Deployment. Goal: Prepare ROLA for production build, testing, release, monitoring, and rollback.

Requirements:

- Create CI/CD workflows for build, test, and release.
- Configure Android SDK setup, Gradle cache, dev/staging/production flavors, secure Firebase config injection, and signed AAB generation.
- Prepare release secrets, signing strategy, Crashlytics mapping upload, Firebase deploy, and Google Play track rollout.
- Create production checklist, security checklist, maintenance strategy, and long-term update roadmap.
- Document environment configs, monitoring, backup, staged rollout, and model/content rollback.

Restriction / Output Note: Final release still requires real Firebase configs, secrets, Play Console setup, and device validation.

Module 22: Enterprise Education Platform

Source: Recovered from project documentation

Clean Development Prompt:

You are an expert EdTech enterprise platform architect specializing in school deployments, roles, classroom workflows, LMS, Firebase security, and Android dashboards. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Previous modules completed as prerequisites. Now implement Module 22: Enterprise Education Platform. Goal: Add institution, classroom, teacher, student, assignment, analytics, LMS, and administration capability.

Requirements:

- Create enterprise layer with RoleManager, InstitutionManager, ClassroomLearningMode, AssignmentManager, AITeachingAssistant, InstitutionAnalyticsEngine, and LMSConnector.
- Model Super Admin, Institution Admin, Teacher, Student, and Parent roles.
- Create Room tables and Firestore collections for institutions, roles, departments, courses, classes, members, assignments, submissions, sessions, analytics, reports, LMS, and audit logs.
- Build teacher dashboard, student dashboard, classroom learning mode, assignment workflow, and LMS integration strategy.
- Ensure organization isolation, Firebase custom claims, rules, audit logs, parent access limits, performance indexes, and enterprise tests.

Restriction / Output Note: Keep enterprise features around the existing ROLA learning platform.

Module 23: Global AI Education Network

Source: Recovered from project documentation

Clean Development Prompt:

You are an expert global EdTech, multi-tenant SaaS, marketplace, collaboration, community, translation, and cloud architecture engineer. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Previous modules completed as prerequisites. Now implement Module 23: Global AI Education Network. Goal: Create a global multi-institution education network for ROLA.

Requirements:

- Create global network layer with TenantManager, GlobalInstitutionNetwork, GlobalNetworkRepository, and GlobalTranslationBridge.
- Support shared knowledge resources, marketplace listings, AR lessons, 3D models, quizzes, research, pathways, and simulations.
- Create collaboration rooms, project workspaces, shared learning sessions, community feed, discussions, and learning groups.
- Design global collections and tenant-scoped data paths.
- Implement knowledge sharing workflow with visibility, license, access, and import controls.
- Use CDN, regional deployments, tenant isolation, privacy-preserving analytics, moderation queues, and global recommendation expansion.

Restriction / Output Note: Do not expose private tenant data through global marketplace/community features.

Module 24: Autonomous AI Teacher and Curriculum Generator

Source: Recovered from project documentation

Clean Development Prompt:

You are an expert AI teacher, curriculum generation, adaptive instruction, assessment design, and education platform engineer. Continue developing the Reality Overlay Learning App (ROLA) based on the existing implementation. Completed modules/context: Previous modules completed as prerequisites. Now implement Module 24: Autonomous AI Teacher and Curriculum Generator. Goal: Create an autonomous AI teacher platform that generates curriculum, lessons, activities, assessments, and teaching plans.

Requirements:

- Create AITeacherEngine with CurriculumGenerator, LessonGenerator, AssessmentGenerator, CurriculumQualityValidator, AdaptiveTeachingEngine, ClassroomAssistant, and AITeacherRepository.
- Use teacher input: subject, topic, grade, objective, duration, and student level.
- Retrieve knowledge graph context and generate modules, lessons, activities, and assessments.
- Validate scientific accuracy, level alignment, objective coverage, and consistency.
- Support teacher review workflow: draft -> pending review -> approved/rejected -> distributed.
- Create Room tables, Firestore collections, core indexes, AI teacher dashboard, tests, and future server-side teacher agents.

Restriction / Output Note: Keep AI teacher platform separate enough that current school workflows remain stable.

Recommended Usage

- Use one module prompt at a time in chronological order.
- Before implementing a module, verify the current project state and avoid rewriting unrelated completed modules.
- When exact original prompt text is required, replace reconstructed modules with exported chat history if it becomes available.
- Keep generated code aligned with Clean Architecture, MVVM, Hilt, Kotlin Coroutines, Flow/StateFlow, Room, Firebase, ARCore, TensorFlow Lite, and Material 3 standards.